CyberSentry — Project Guide and Presentation Notes

An Intelligent Cybersecurity Expert System for Analysing Suspicious Messages

Purpose of this document. This guide supports the first project presentation. It explains what CyberSentry is, why it was built, how it works end to end, which Prolog concepts it uses, and what to demonstrate. It can be read from beginning to end or used as a reference for individual sections.

1. Project Overview

CyberSentry is an intelligent cybersecurity expert system that analyses suspicious messages (SMS, WhatsApp, email, chat) and explains the danger in simple language.

The system has two main parts:

1. **A Prolog rule-based threat engine.** It detects known scam indicators, classifies risk, identifies threat categories, and produces evidence and safety advice.
2. **A Flask web application.** It provides a message scanner dashboard and an AI security assistant powered by Gemini.

The system is designed with Ugandan users in mind and focuses on threats such as mobile money fraud, fake job offers, impersonation, phishing, and account takeover.

One-Sentence Summary

> *"CyberSentry is a web-based cybersecurity expert system that uses Prolog rules to detect scam indicators in a message, score its risk, and explain why it is dangerous, then uses Gemini AI to discuss the findings with the user in natural language."*

2. Problem Statement

Many people receive fraudulent messages every day, for example:

- "You have won a prize — send your PIN to claim it."
- "Your account has been suspended. Click this link immediately."
- "I am from your bank. Share your OTP to verify your account."
- "Pay a registration fee to get this job."

These messages rely on **social engineering** — pressure, fear, greed, and impersonation — to trick people into sharing credentials or sending money.

Problem. Ordinary users often cannot tell a scam message from a legitimate one, and they do not know what to do next.

Solution. CyberSentry gives users a fast, rule-based second opinion. It flags warning signs, explains them, assigns a risk level, and gives practical advice.

3. Objectives

1. Represent cybersecurity knowledge as Prolog facts and rules.
2. Detect suspicious indicators in free-text messages.
3. Combine indicators into threat categories.
4. Calculate a risk score and classify the message as LOW, MEDIUM, or HIGH risk.
5. Provide an explanation for every detected indicator.
6. Give general and category-specific safety advice.
7. Provide a conversational AI assistant that explains the Prolog findings.
8. Present everything through a clean web interface.

4. Key Features

Feature	Description
Suspicious message scanner	Accepts pasted text and returns a full risk report
Rule-based detection	Uses 194 phrase patterns covering 19 indicator types
Risk scoring	Adds weights for detected indicators and applies thresholds
Threat categorisation	Maps indicators to 14 threat categories
Evidence and explanation	Explains why each indicator was flagged
Safety advice	Gives general and category-specific recommendations
AI security assistant	Gemini explains findings and answers cybersecurity questions
Safe rendering	AI text is rendered as text, not raw HTML (prevents XSS)
Input validation	Length limits and type checks on both client and server

5. Technology Stack

Technology	Role
SWI-Prolog	Rule-based reasoning engine and knowledge representation
PySwip	Python–Prolog bridge; lets Python query Prolog predicates
Python 3.14	Application logic and integration
Flask	Web server and REST API
Gemini (google-genai)	Conversational AI for explanations and cybersecurity chat
python-dotenv	Loads the Gemini API key from .env
HTML / CSS / JavaScript	Frontend dashboard and chat interface
JSON	Data exchange between browser, Flask, and Gemini
Python virtual environment (venv)	Isolated dependencies

Not used. There is no database in the current version. Chat history is kept in browser memory only.

6. System Architecture

CyberSentry is organised into four layers, shown in Figure 1.

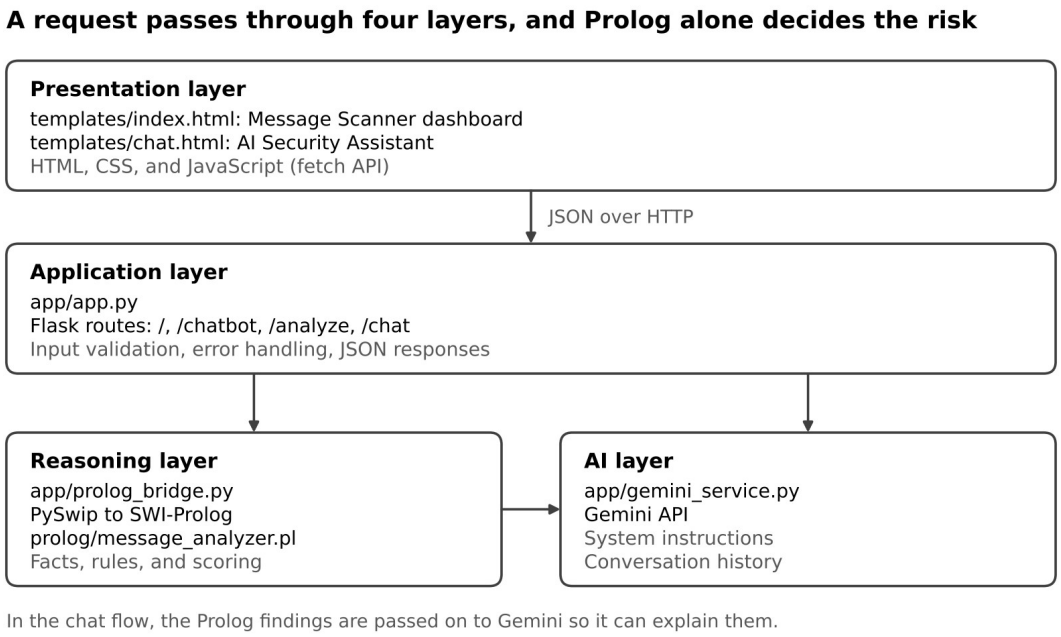


Figure 1. System architecture: four layers

The presentation layer talks to the Flask application over HTTP, and the application calls the reasoning layer and, for chat, the AI layer.

Layer Responsibilities

- **Presentation:** collects the message, sends it to the server, and displays the result.
- **Application:** validates input, calls the right service, and returns JSON.
- **Reasoning:** the Prolog expert system performs the actual threat analysis.
- **AI:** Gemini turns the structured findings into a natural, conversational explanation.

7. Project Structure

```
CyberSentry/
├── app/
│   ├── app.py              # Flask application and API routes
│   ├── prolog_bridge.py      # Python <-> Prolog bridge (PySwip)
│   └── gemini_service.py     # Gemini AI integration
├── prolog/
│   ├── message_analyzer.pl    # THE ACTIVE knowledge base + inference engine
│   └── examples/
│       └── knowledge_base_example.pl   # Earlier sample KB (NOT loaded)
├── templates/
│   ├── index.html           # Message scanner dashboard
│   └── chat.html            # AI assistant chat interface
├── database/               # Reserved (currently empty)
├── static/                 # Reserved (currently empty)
├── tests/                  # Reserved (currently empty)
├── venv/                    # Python virtual environment
├── .env                    # GEMINI_API_KEY (secret; never commit)
├── .gitignore
└── CyberSentry_Project_Guide.md
```

Important. prolog/message_analyzer.pl is the only Prolog file loaded by the application. The file in prolog/examples/ is kept as an example and is **not** part of the running system.

8. End-to-End Workflow

8.1 Message Scanner Workflow (/analyze)

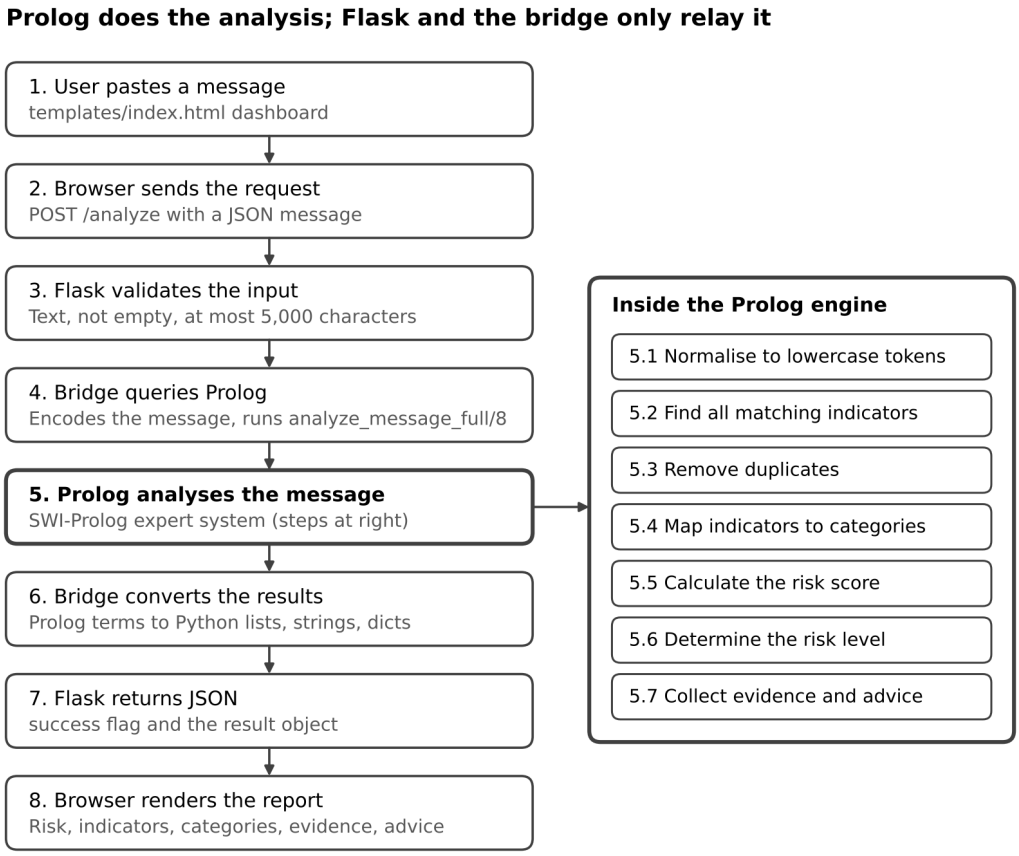


Figure 2. Message Scanner workflow

The detailed steps are as follows.

1. The user pastes a suspicious message into the dashboard (templates/index.html).
2. JavaScript sends a POST /analyze request with JSON: { "message": "..." }.
3. Flask validates the message:
   - it must be text,
   - it must not be empty,
   - it must not exceed 5,000 characters (enforced again in the bridge).
4. analyzer.analyze(message) is called from app/prolog_bridge.py.
5. The bridge:
   - safely encodes the message as a Prolog string using json.dumps,
   - builds a Prolog query: analyze_message_full(Message, I, R, S, C, E, A, CA),
   - executes it with PySwip.
6. Prolog:

- normalises the text into lowercase tokens,
- finds all matching indicators,
- removes duplicates,
- maps indicators to threat categories,
- calculates the risk score,
- determines the risk level,
- collects evidence and advice.

7. The bridge converts Prolog terms into Python data (lists, strings, dictionaries).
8. Flask returns JSON: { "success": true, "result": { ... } }.
9. JavaScript renders:
   - the risk level and score,
   - detected indicators as chips,
   - threat categories,
   - evidence with explanations,
   - general and category-specific advice.

8.2 AI Assistant Workflow (/chat)

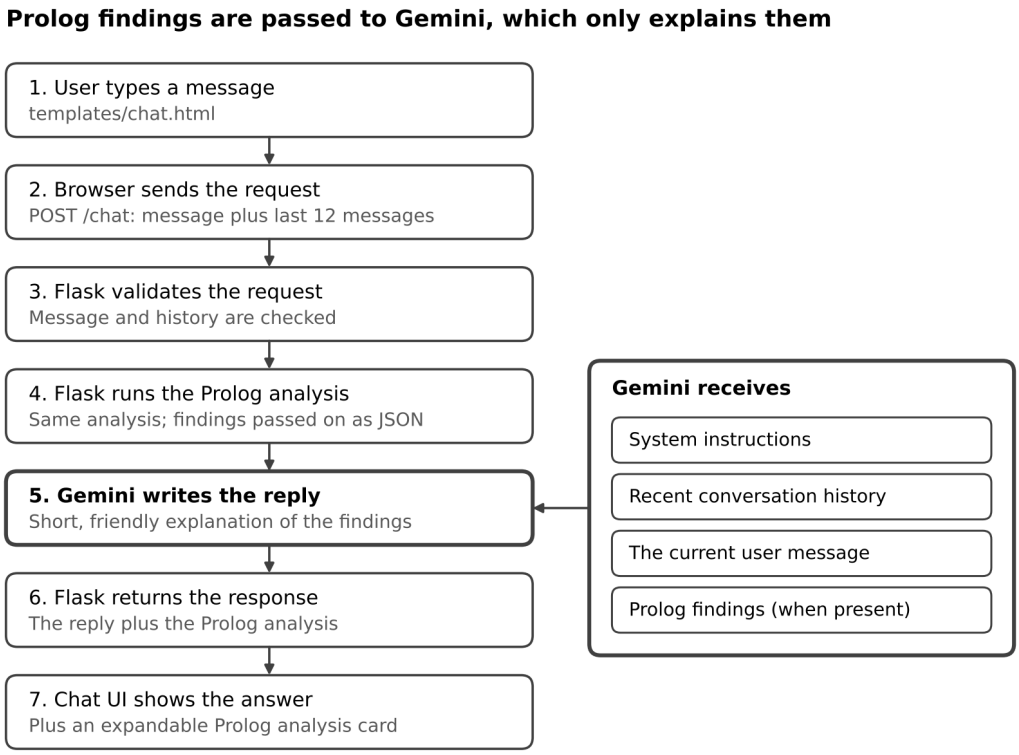


Figure 3. AI Assistant workflow

The detailed steps are as follows.

1. The user types a message in templates/chat.html.
2. The browser sends POST /chat with the current message and recent history (the last 12 messages).
3. Flask validates the message and history.
4. Flask runs the **same Prolog analysis** on the current message.
5. If indicators were found, the findings are serialised to JSON and passed to Gemini.
6. Gemini receives:
   - system instructions (cybersecurity assistant, Uganda focus, safety rules),
   - recent conversation history,
   - the current user message,
   - the Prolog findings (when present).
7. Gemini generates a short, friendly explanation.
8. Flask returns the reply and the analysis.
9. The chat UI displays the reply and an expandable "Prolog Threat Analysis" card.

Key design rule. Prolog is the authority for detection and risk. Gemini only explains; it is explicitly instructed not to invent or override findings.

9. The Prolog Expert System

This is the core of the project. The knowledge base is prolog/message_analyzer.pl, organised into 11 sections.

9.1 Why Prolog?

Prolog is a **declarative logic programming language**. Instead of writing step-by-step instructions ("loop through words, check this, increment that"), the programmer describes **facts** and **rules**, and Prolog's inference engine works out what is true.

This makes Prolog a natural fit for an **expert system**, because:

- cybersecurity knowledge is naturally expressed as rules;
- adding a new scam pattern is just adding a fact;
- the same rules can explain their reasoning (evidence);
- the logic is separate from the web interface.

9.2 Knowledge Representation: Facts

A **fact** is something the system knows to be true. Examples from the project:

```
indicator_pattern(requests_pin, "your pin").
indicator_pattern(requests_password, "password").
indicator_pattern(suspicious_link, "click this link").
```

```
indicator_pattern(urgent_action, "immediately").
indicator_weight(requests_pin, 10).
threat_category(requests_pin, credential_theft).
```

These facts read as follows:

- The pattern `your pin` is an indicator of `requests_pin`.
- An indicator of type `requests_pin` has weight 10.
- `requests_pin` belongs to the threat category `credential_theft`.

9.3 Knowledge Representation: Rules

A **rule** has a head and a body:

```
head :- body.
```

It means: “head is true if body is true.” Example from the risk classifier:

```
message_risk(Indicators, high) :-
    member(Indicator, Indicators),
    critical_indicator(Indicator),
    !.
```

This reads: “The risk is high if the list of indicators contains at least one critical indicator.”

9.4 Predicates and Arity

A **predicate** is identified by its name and its number of arguments (its **arity**):

Predicate	Description
analyze_message/2	Message in, indicator list out
analyze_message/4	Message, indicators, risk, advice
analyze_message_extended/7	Adds score, categories, evidence
analyze_message_full/8	Adds category-specific advice
message_risk/2	Indicators in, risk out
calculate_risk_score/2	Indicators in, numeric score out

Prolog allows the same name with different arity; these are treated as different predicates.

9.5 Atoms, Strings, Variables, and Compound Terms

- **Atoms:** `requests_pin`, `high`, `credential_theft`.
- **Strings:** `"your pin"`, `"HIGH RISK: ..."`.
- **Variables:** `Message`, `Indicators`, `Risk` (they start with an uppercase letter).

- **Compound terms:** evidence(requests_pin, "The message requests a PIN...").

9.6 Unification

Unification is Prolog's pattern-matching mechanism. It tries to make two terms equal by binding variables.

```
?- indicator_pattern(requests_pin, Pattern).
Pattern = "your pin" ;
Pattern = "pin number" ;
...
```

Prolog unifies Pattern with each matching fact in turn.

9.7 Backtracking

When a goal can be satisfied in more than one way, Prolog can **backtrack** to find alternative solutions. In this project, findall/3 deliberately triggers backtracking to collect **all** matching indicators:

```
findall(
   Indicator,
   contains_indicator_tokens(MessageTokens, Indicator),
   FoundIndicators
)
```

9.8 The Cut Operator (!)

The **cut** ! tells Prolog: "stop looking for alternative solutions to this goal." It is used in two important places.

Phrase matching. Once a phrase is found, there is no need to search other positions:

```
phrase_in_tokens(Phrase, Tokens) :-
   append(_, Remaining, Tokens),
   append(Phrase, _, Remaining),
   !.
```

Risk classification. Once a message is HIGH, Prolog does not continue to MEDIUM or LOW:

```
message_risk(Indicators, high) :-
   member(Indicator, Indicators),
   critical_indicator(Indicator),
   !.
```

Without the cut, the query could produce multiple risk answers. With it, the first matching risk level is committed.

9.9 List Processing

The project uses standard SWI-Prolog list predicates:

Predicate	Purpose
append/3	Checks whether a phrase occurs inside a token list
member/2	Checks whether an indicator is in a list
findall/3	Collects all solutions into a list
sort/2	Sorts and removes duplicates
sum_list/2	Adds indicator weights
length/2	Counts list elements

Recursion is hidden inside these library predicates. For example, append/3 is recursive: it splits a list at every possible point.

9.10 Text Normalisation and Phrase Matching

Normalisation (normalize_text/2):

1. Converts the message to lowercase with string_lower/2.
2. Splits it into tokens with split_string/4, using spaces and punctuation as separators.

Separators include spaces, tabs, newlines, common ASCII punctuation, quotes, dashes, curly quotes, ellipsis, bullets, and non-breaking spaces. This makes matching case-insensitive and punctuation-insensitive.

Phrase matching (phrase_in_tokens/2):

```
phrase_in_tokens(Phrase, Tokens) :-
  append(_, Remaining, Tokens),
  append(Phrase, _, Remaining),
  !.
```

This says: "There exists a prefix of Tokens such that Phrase is a prefix of the remaining suffix." In plain English, **the phrase occurs contiguously somewhere in the token list**.

Performance note. The message is normalised **once** in analyze_message/2 and then reused for all 194 patterns. This avoids re-tokenising the same message hundreds of times.

9.11 Section-by-Section Breakdown of message_analyzer.pl

Section	Contents
1. Text processing	normalize_text/2, phrase_in_tokens/2, contains_indicator/2, contains_indicator_tokens/2

Section	Contents
2. Cybersecurity indicators	194 indicator_pattern/2 facts across 19 indicator types
3. Indicator explanations	indicator_description/2: a human-readable reason for each indicator
4. Threat categories	threat_category/2: maps indicators to 14 categories
5. Message analysis	analyze_message/2, analyze_categories/2, analyze_evidence/2
6. Risk weighting	indicator_weight/2, calculate_risk_score/2
7. Risk classification	critical_indicator/1, message_risk/2
8. Risk recommendations	recommendation/2 for high, medium, and low risk
9. Category advice	category_advice/2, category_recommendations/2
10. Complete output	analyze_message/4, analyze_message_extended/7, analyze_message_full/8
11. System information	system_name/1, system_description/1, system_version/1

9.12 The 19 Indicator Types

1. requests_pin
2. requests_password
3. requests_otp
4. requests_secret
5. suspicious_link
6. urgent_action
7. threatening_language
8. suspicious_payment
9. unexpected_reward
10. fake_job_offer
11. investment_scam
12. impersonation
13. mobile_money_fraud
14. account_takeover
15. malware_threat
16. remote_access_request
17. requests_personal_information
18. delivery_scam
19. emotional_manipulation

9.13 The 14 Threat Categories

credential_theft, phishing, social_engineering, financial_fraud, reward_scam, employment_fraud, investment_fraud, impersonation_fraud, mobile_money_fraud, account_compromise, malware, unauthorized_access, identity_theft, delivery_fraud.

9.14 Risk Weighting and Classification

Each indicator has a weight:

Level	Weight	Indicators
Critical	10	PIN, password, OTP, secret key
High	8	Suspicious payment, account takeover, remote access, malware
Medium	5	Personal information
Lower	2–4	Links, impersonation, mobile money, investment, job offers, delivery, threats, rewards, urgency, emotional manipulation

The score is the sum of the weights of all distinct detected indicators:

```
calculate_risk_score(Indicators, Score) :-
  findall(
    Weight,
    (member(Indicator, Indicators), indicator_weight(Indicator, Weight)),
    Weights
  ),
  sum_list(Weights, Score).
```

The classification rules are applied in this order (see Figure 4):

1. Any **critical** indicator gives **HIGH**.
2. Otherwise, a score of 7 or more gives **HIGH**.
3. Otherwise, a score of 3 or more gives **MEDIUM**.
4. Otherwise the result is **LOW**.

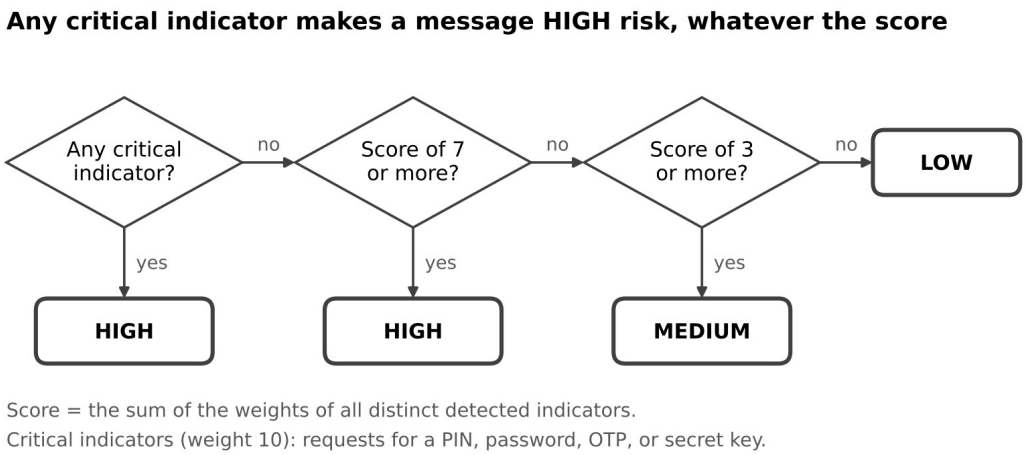


Figure 4. Risk classification rules

This is rule-based **forward reasoning**: from detected facts (indicators), the system derives a conclusion (the risk level).

9.15 Worked Example (for the Demonstration)

Message:

> *"You have won a prize! Send your PIN immediately to claim your reward."*

Step	Action	Result
1	Normalise the message	Lowercase tokens
2	Detect indicators	requests_pin (pattern "your pin"), unexpected_reward (pattern "you have won"), urgent_action (pattern "immediately")
3	Map to categories	credential_theft, reward_scam, social_engineering
4	Calculate the score	10 (PIN) + 2 (reward) + 2 (urgency) = **14**
5	Classify the risk	requests_pin is critical, so the risk is **HIGH**
6	Produce outputs	3 explanations (evidence), HIGH-risk general advice, category-specific advice

This exact example can be run from the command line:

```
cd /home/ssali/Desktop/SLAU/LogicProgramming/CyberSentry
swipl -q -g "consult('prolog/message_analyzer.pl'),
  analyze_message_full(
    \"You have won a prize! Send your PIN immediately to claim your reward\",
    I, R, S, C, E, A, CA),
  write(indicators=I), nl,
  write(risk=R), nl,
  write(score=S), nl,
```

```
write(categories=C), nl,
write(evidence=E), nl,
write(advice=A), nl,
write(category_advice=CA), nl,
halt"
```

9.16 The Three Output Predicates

- analyze_message(Message, Indicators, Risk, Advice): simple, bridge-compatible output.
- analyze_message_extended(Message, Indicators, Risk, Score, Categories, Evidence, Advice): adds score, categories, and evidence.
- analyze_message_full(Message, Indicators, Risk, Score, Categories, Evidence, Advice, CategoryAdvice): the complete output used by the web application.

The Python bridge calls analyze_message_full/8.

10. Prolog Concepts Used: Quick Reference

Concept	Meaning	Where it appears
Fact	Something known to be true	indicator_pattern/2, indicator_weight/2, etc.
Rule	Conclusion that depends on conditions	message_risk/2, calculate_risk_score/2
Predicate	Named relation with arguments	analyze_message/2, contains_indicator/2
Arity	Number of arguments	/2, /4, /8
Atom	Lowercase symbolic constant	requests_pin, high
String	Text in double quotes	"your pin"
Variable	Placeholder, starts uppercase	Message, Indicators
Compound term	Functor with arguments	evidence(Indicator, Description)
Unification	Pattern matching and variable binding	indicator_pattern(requests_pin, Pattern)
Backtracking	Search for alternative solutions	Inside findall/3, member/2
Cut (!)	Commit to the first solution	phrase_in_tokens/2, message_risk/2
List	Ordered collection	Indicators, Tokens
Recursion	Predicate defined in terms of itself	Inside append/3, member/2
findall/3	Collect all solutions	analyze_message/2, analyze_evidence/2

Concept	Meaning	Where it appears
sort/2	Sort and remove duplicates	Indicator and category lists
append/3	List concatenation and splitting	Phrase matching
member/2	List membership	Category and weight lookups
sum_list/2	Sum a numeric list	Risk score
Modules	Load standard libraries	use_module(library(lists)), library(apply)
Consult	Load a Prolog file	prolog_bridge.py

11. The Python–Prolog Bridge (app/prolog_bridge.py)

11.1 Purpose

PrologBridge is the adapter between Python and SWI-Prolog.

11.2 Initialisation

```
self.prolog = Prolog()
self.prolog.consult(str(analyzer_path))
```

This starts a Prolog engine and loads prolog/message_analyzer.pl.

11.3 Safe Message Encoding

The message is converted into a valid Prolog string with:

```
safe_message = json.dumps(message, ensure_ascii=False)
```

This prevents broken queries when the message contains quotes, backslashes, or Unicode characters.

11.4 Querying Prolog

```
query = f"analyze_message_full({safe_message}, I, R, S, C, E, A, CA)"
results = list(self.prolog.query(query))
```

Variable	Meaning
I	Indicators
R	Risk
S	Score
C	Categories

Variable	Meaning
E	Evidence
A	General advice
CA	Category advice

11.5 Converting Prolog Values

- _decode/1 turns PySwip values (including bytes and nested lists) into normal Python values.
- _parse_evidence/1 converts Prolog terms such as evidence(requests_pin, "explanation") into dictionaries:

```
{ "indicator": "requests_pin", "explanation": "..." }
```

11.6 Result Structure

```
{
  "message": "...",
  "indicators": ["requests_pin", "urgent_action"],
  "risk": "high",
  "score": 12,
  "categories": ["credential_theft", "social_engineering"],
  "evidence": [
    { "indicator": "requests_pin", "explanation": "..." }
  ],
  "advice": "HIGH RISK: ...",
  "category_advice": ["Credential safety: ..."]
}
```

12. The Web Layer (app/app.py)

12.1 Routes

Route	Method	Purpose
/	GET	Message scanner dashboard
/chatbot	GET	AI assistant page
/analyze	POST	Run Prolog analysis only
/chat	POST	Run Prolog analysis and Gemini reply

12.2 Validation

- The message must be a non-empty string.

- The maximum length is 5,000 characters.
- Chat history must be a list; invalid entries are filtered out.
- Only the last 12 history messages are kept.
- The request body size is capped at 32 KB.

12.3 Error Handling

- `ValueError` returns HTTP 400 with a helpful message.
- Any other exception is logged and returns HTTP 500 with a generic message (no internal details are leaked).

13. The AI Layer (`app/gemini_service.py`)

13.1 Configuration

- Loads `GEMINI_API_KEY` from `.env` using `python-dotenv`.
- Creates a Gemini client.
- The model is configured in `MODEL` (currently `gemini-3.5-flash-lite`).

13.2 System Instruction

Gemini is instructed to act as **CyberSentry**, a professional cybersecurity assistant focused on Uganda. It must:

- explain threats and scams clearly,
- keep responses short and friendly,
- never ask for PINs, passwords, or OTPs,
- never invent security findings,
- never override the Prolog risk classification.

13.3 Conversation History

`chat_with_gemini()` builds a `contents` list from recent history, then appends:

- the current user message,
- the Prolog findings (when indicators exist).

13.4 Design Rationale

Prolog gives **deterministic, explainable detection**. Gemini gives **natural-language explanation**. Combining them provides both accuracy and usability.

14. The Frontend

14.1 Message Scanner (templates/index.html)

- Dark cybersecurity dashboard layout.
- Textarea with a 5,000-character limit.
- fetch("/analyze") sends JSON.
- Renders risk level, score, indicators, categories, evidence, and advice.
- Uses textContent and createElement to insert data safely.

14.2 AI Assistant (templates/chat.html)

- Chat interface with suggested questions.
- Sends the message and history to /chat.
- Renders a limited, safe subset of Markdown.
- Displays a "Prolog Threat Analysis" card under each answer that had findings.
- Keeps history in JavaScript memory only (it resets on refresh).

14.3 XSS Protection

AI responses are **not** inserted with innerHTML. The code builds DOM nodes and uses textContent, so malicious HTML in a message or an AI reply is displayed as plain text.

15. Security Considerations

Concern	How it is handled
API key leakage	Loaded from .env; .env is listed in .gitignore
Prolog injection	The message is JSON-encoded before being placed in a query
XSS	The frontend uses textContent, not raw HTML insertion
Oversized input	5,000-character limit and 32 KB request cap
Error information leakage	Generic 500 error; details appear only in server logs
Credential safety	Gemini is instructed never to request PINs, passwords, or OTPs
Untrusted AI output	Prolog remains the authority; the AI cannot override findings

Development-only note. The Flask application currently runs with debug=True, which must be turned off before any public deployment.

16. Testing and Verification

16.1 Testing the Prolog Engine Directly

```
cd /home/ssali/Desktop/SLAU/LogicProgramming/CyberSentry
swipl -q -g "consult('prolog/message_analyzer.pl'),
  analyze_message_full(\"Share your OTP immediately\", I, R, S, C, E, A, CA),
  write(I), nl, write(R), nl, write(S), nl, halt"
```

16.2 Testing the Python Bridge

```
cd /home/ssali/Desktop/SLAU/LogicProgramming/CyberSentry/app
../venv/bin/python -c "
from prolog_bridge import analyzer
r = analyzer.analyze('Share your OTP immediately')
print(r['risk'], r['score'], r['indicators'])
"
```

16.3 Example Test Cases

Message	Expected result
You have won a prize! Send your PIN immediately.	HIGH, score 14, credential theft, reward scam, urgency
Share your OTP immediately or your account will be suspended.	HIGH, OTP request, threat, urgency
Your account has been suspended. Click this link to verify your account here.	HIGH, phishing, threat, urgency
Pay a processing fee to claim your prize.	HIGH, financial fraud, reward scam
The shipping pin is on the map.	LOW, score 0 (no false positive)
He said "send money" and it's urgent.	HIGH, quoted phrase detected correctly
Hello, how are you today?	LOW, score 0

16.4 Manual Web Testing

1. Start the application (see Section 17).
2. Open http://127.0.0.1:5000/.
3. Paste each message above and confirm the risk and indicators.
4. Open http://127.0.0.1:5000/chatbot and ask a cybersecurity question.
5. Confirm that the Prolog analysis card appears when indicators are found.

17. Setup and How to Run

17.1 Requirements

- SWI-Prolog installed (`swipl`)
- Python 3.x
- The project `venv` (or install the packages listed below)

17.2 Python Packages Used

```
flask
pyswip
google-genai
python-dotenv
```

17.3 Environment File

Create a `.env` file in the project root:

```
GEMINI_API_KEY=your_key_here
```

Do **not** commit this file.

17.4 Running the Application

```
cd /home/ssali/Desktop/SLAU/LogicProgramming/CyberSentry
venv/bin/python app/app.py
```

Alternatively:

```
cd /home/ssali/Desktop/SLAU/LogicProgramming/CyberSentry/app
../venv/bin/python app.py
```

Then open:

- Message Scanner: `http://127.0.0.1:5000/`
- AI Assistant: `http://127.0.0.1:5000/chatbot`

18. Suggested First-Presentation Outline

Use the following as the slide sequence. Each slide has a short line to say aloud.

Slide	Title	Content	Suggested line to say
1	Title	**CyberSentry: An Intelligent Cybersecurity Expert System**	"My project is CyberSentry, a web application that uses Prolog rules and AI to detect and explain suspicious messages."

19. Live Demonstration Script

Demo 1: Obvious Scam

Paste:

> *"Congratulations! You have won a prize. Send your PIN immediately to claim your reward."*

Point out:

- HIGH risk
- score 14
- indicators: PIN request, unexpected reward, urgency
- categories: credential theft, reward scam, social engineering
- explanations and advice

Demo 2: Phishing with a Threat

Paste:

> *"Your account has been suspended. Click this link immediately to verify your account here."*

Point out:

- threat, phishing link, and urgency
- HIGH risk
- category-specific advice

Demo 3: Benign Message

Paste:

> *"The shipping pin is on the map. See you at 5."*

Point out:

- LOW risk
- score 0
- this shows the detector is not simply flagging the word "pin"

Demo 4: AI Assistant

Open the chat page and ask:

> *"Someone asked me to share my OTP. Is that safe?"*

Point out:

- Gemini's natural-language answer
- the Prolog analysis card showing `requests_otp`

20. Limitations

1. **Pattern-based detection.** The engine matches known phrases; it does not fully understand meaning.
2. **Exact contiguous phrases.** “account suspended” matches, but unusual wording may not.
3. **English-focused.** There is no Luganda or multilingual matching yet.
4. **False positives and negatives are possible.** Single words can appear in harmless contexts, and new scam wording can be missed.
5. **No chat persistence.** History resets on page refresh.
6. **No database.** There is no storage of analyses, users, or reports.
7. **No authentication or rate limiting.** The API is open on localhost.
8. **Development server settings.** `debug=True` must be disabled for deployment.
9. **AI is not authoritative.** Gemini explains findings but can still phrase things imperfectly; Prolog remains the source of detection.

21. Future Work

- Expand the indicator patterns and add synonyms and typo handling.
- Add URL and domain analysis (shortened links, blacklists).
- Add multilingual support, including Luganda.
- Add a database for chat history and saved reports.
- Add user accounts and rate limiting.
- Add automated tests (`plunit` for Prolog, `pytest` for Python).
- Add confidence levels instead of only three risk levels.
- Deploy with a production server (for example, Gunicorn) and `debug=False`.

22. Likely Lecturer Questions and Answers

Q: Why did you use Prolog instead of Python for the detection logic?

A: The detection logic is knowledge-based. Prolog lets me express facts and rules declaratively, add new scam patterns easily, and explain conclusions. Python handles the web and AI integration.

Q: What is the difference between a fact and a rule?

A: A fact is unconditionally true, for example `indicator_pattern(requests_pin, "your pin")`. A rule is true if its body is true, for example `message_risk(Indicators, high) :- member(...), critical_indicator(...), !.`

Q: What is unification?

A: It is Prolog’s pattern-matching mechanism. It binds variables to values so that two terms become equal, for example matching `Indicator` to each indicator pattern.

Q: What is backtracking?

A: When a goal has several possible solutions, Prolog can return to an earlier choice point and try another. `findall/3` uses this to collect every matching indicator.

Q: What does the cut ! do?

A: It prevents Prolog from trying alternative solutions after a match. It makes phrase matching and risk classification deterministic.

Q: How is the risk score calculated?

A: Each detected indicator has a weight, and the weights are summed. Any critical indicator makes the message HIGH; otherwise a score of 7 or more is HIGH, 3 or more is MEDIUM, and the rest are LOW.

Q: How do you prevent Prolog injection?

A: The message is encoded as a JSON string before being inserted into the Prolog query, so quotes and special characters cannot break the query.

Q: How do you prevent XSS from AI output?

A: The frontend builds DOM nodes and uses `textContent` instead of inserting raw HTML.

Q: What is the role of Gemini?

A: Gemini provides conversational explanations and general cybersecurity guidance. It does not perform detection and is instructed not to override Prolog.

Q: How would you test this system?

A: With known scam and benign messages, checking the expected indicators, scores, and risk levels. Automated tests can use `plunit` for Prolog and `pytest` for the Python bridge.

Q: What are the limitations?

A: It is pattern-based, English-focused, and can miss paraphrased or obfuscated scams. It also has no database, authentication, or rate limiting yet.

23. Glossary

Term	Meaning
Expert system	A program that uses a knowledge base and inference rules to give expert-level advice
Knowledge base	The collection of facts and rules
Inference engine	The part of Prolog that applies rules to facts
Fact	A statement known to be true
Rule	A conditional statement: the head is true if the body is true
Predicate	A named relation, for example `indicator_pattern/2`

Term	Meaning
Arity	The number of arguments a predicate takes
Atom	A lowercase symbolic constant, for example `high`
Variable	A placeholder that can be bound to a value, for example `Indicators`
Unification	Matching terms and binding variables
Backtracking	Searching for alternative solutions
Cut	!: commits to the current solution
`findall`	Collects all solutions into a list
Forward reasoning	Deriving conclusions from known facts
Indicator	A warning sign found in a message
Threat category	A group of related indicators, for example credential theft
Risk score	A numeric total of indicator weights
PySwip	A Python library that connects to SWI-Prolog
Flask	A lightweight Python web framework
Gemini	Google's generative AI model used for conversation
XSS	Cross-site scripting; prevented here by safe DOM rendering
Prompt injection	Attempting to manipulate an AI through crafted input

24. Final Summary

CyberSentry combines:

- **Prolog** for declarative, explainable threat detection;
- **Python and Flask** for the application and API;
- **PySwip** to connect the two;
- **Gemini** for natural-language explanations;
- **HTML, CSS, and JavaScript** for a usable interface.

The result is a working expert system that can take a suspicious message, identify known scam indicators, classify the risk, explain the reasoning, and advise the user, while an AI assistant helps them understand it in conversation.

Closing line for the presentation:

> *"CyberSentry demonstrates how logic programming can provide transparent, rule-based security analysis, and how that analysis can be combined with modern AI to make cybersecurity advice accessible to ordinary users."*